

A EXEMPLAR QUERIES

```
-- Extract keywords from a content field
SELECT id, s'Extract the main keywords from {content}' AS
↪ extracted_keywords
FROM documents;
-- Create a semantic summary from multiple fields
SELECT s'A summary of the event based on {title} and
↪ {description}'
AS event_summary FROM news_articles;
```

Figure 10: Example of Semantic Projection

```
-- Filter rows based on the sentiment of a specific field
SELECT * FROM product_reviews
WHERE s'The {review_text} expresses positive sentiment';
```

Figure 11: Example of Semantic Filter

```
-- Analyze the language style of the conference paper title
WITH title_style AS (
  SELECT Title, s'Analyze the language style features of paper
  ↪ title {Title}.
  Describe the key characteristics such as: formality level,
  technical terminology usage, length, structure, and word
  ↪ choice.'
  AS style_features
  FROM Paper WHERE ConferenceId != 0 AND Title IS NOT NULL
)
SELECT sem_agg(s'Based on these conference paper titles {Title}
and their style analyses {style_features}, summarize the overall
language style and terminology characteristics of conference
↪ paper titles.')
FROM title_style;
```

Figure 12: Example of Semantic Aggregate

```
-- Join two tables based on semantic similarity of their text
↪ fields
SELECT * FROM news_articles a JOIN blog_posts b
ON s'the topic of {a.headline} is related to {b.title}';
-- A complex hybrid join with both syntactic and semantic
↪ conditions
SELECT e.employee_id, eh.manager_id
FROM employees e INNER JOIN employee_hierarchy eh
ON e.manager_id = eh.employee_id
AND s'the {e.title} is a technical role and the hierarchy level
↪ {eh.level} < 3';
```

Figure 13: Example of Semantic Join

Algorithm 1: Natural Language Expression Reduction

Input :userInputNLE
Output:processedNLE

```
1 reductionStatus ← ASSESSDBREDUCTION(userInputNLE)
2 if reductionStatus = "not_reducible" then
3   return processedNLE ← cleanedNLE
4 processedNLE ← EXPRESSIONREDUCTION(userInputNLE)
5 return processedNLE
```

Algorithm 2: Expression Exploration and Pairwise Similarity Estimation

Input : $\Sigma = \{\sigma_1, \dots, \sigma_n\}$, sample chunk $\mathcal{D}_1$
Output:Selectivities s , result vectors R , similarity matrix MCC

```
1 for  $i \leftarrow 1$  to  $n$  do
2    $R[i] \leftarrow \text{EXECUTE PREDICATE}(\sigma_i, \mathcal{D}_1)$ ;
3    $S[i] \leftarrow \frac{\text{COUNT TRUE}(R[i])}{|\mathcal{D}_1|}$ ;
4 for  $i \leftarrow 1$  to  $n$  do
5   for  $j \leftarrow i + 1$  to  $n$  do
6      $(TP, TN, FP, FN) \leftarrow \text{CONFUSION COUNTS}(R[i], R[j])$ ;
7      $\text{MCC}[i][j] \leftarrow \text{MATTHEWS}(TP, TN, FP, FN)$ ;
8 return  $S, R, \text{MCC}$ 
```

Algorithm 3: Candidate Path Generation with Dynamic Pruning

Input : $S, \text{MCC}, \Sigma, \mu_{\text{thresh}}$
Output:Candidate paths $\mathcal{P}$

```
1  $p_{\text{sorted}} \leftarrow \text{SORT ASCENDING BY SELECTIVITY}(\Sigma, S)$ ;
2  $\mathcal{P} \leftarrow \{\text{SEQPATH}(p_{\text{sorted}})\}$ ;
3  $\mathcal{F} \leftarrow \emptyset$ ;
4 for  $i \leftarrow 1$  to  $n$  do
5   for  $j \leftarrow i + 1$  to  $n$  do
6     if  $\text{MCC}[i][j] > \mu_{\text{thresh}}$  then
7        $\mathcal{F} \leftarrow \mathcal{F} \cup \{(\sigma_i, \sigma_j)\}$ ;
8 for each  $(\sigma_a, \sigma_b) \in \mathcal{F}$  do
9    $\sigma_a \oplus \sigma_b \leftarrow \text{FUSE}(\sigma_a, \sigma_b)$ ;
10   $\mathcal{P} \leftarrow \mathcal{P} \cup \{\text{SEQPATH}(\text{REPLACE ONCE}(p_{\text{sorted}}, \{\sigma_a, \sigma_b\} \rightarrow \sigma_a \oplus \sigma_b))\}$ ;
11 return  $\mathcal{P}$ 
```

B ALGORITHMS

Algorithm 2 evaluates each predicate ($\sigma_i \in \Sigma$) on the sampled data chunk ($\mathcal{D}_1$) to produce a Boolean result vector $R[i]$, from which the selectivity $S[i]$ is estimated as the fraction of tuples satisfying the predicate (line 1-3). Using these result vectors, the algorithm then computes pairwise predicate similarity by deriving a confusion matrix for each predicate pair and applying the Matthews Correlation Coefficient (MCC) to populate the similarity matrix (line 4-7).

Algorithm 3 generates a set of candidate execution paths with dynamic pruning guided by predicate correlations. Given a set of predicates Σ , the algorithm first sorts all predicates in ascending order of selectivity to construct a baseline sequential execution path (line 1), which is included in the candidate set $\mathcal{P}$ (line 2). It

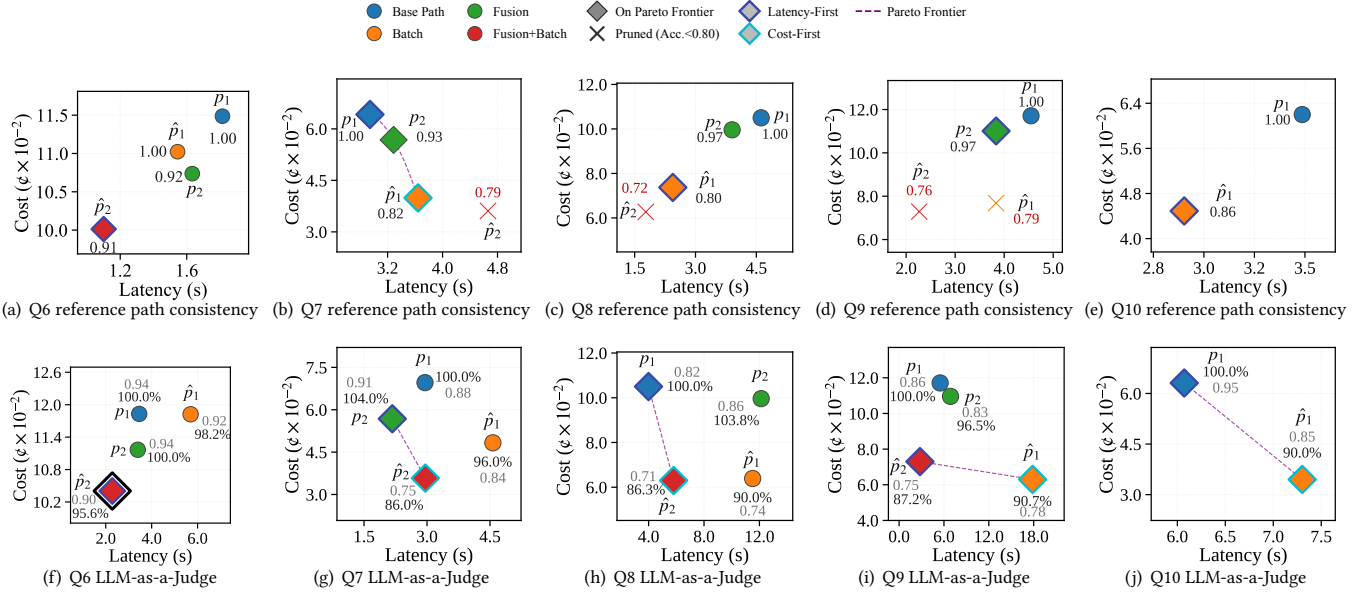


Figure 9: Comparison between reference path consistency and LLM-as-a-Judge scores for Q6-Q10.

Algorithm 4: Path Exploration and Best-Path Selection

Input : Paths $\mathcal{P}$, sample chunk $\mathcal{D}_2$, threshold τ_{acc}

Output : Best path p^*

- 1 Execute reference path to get ground truth G ;
- 2 **for each** $p \in \mathcal{P}$ **do**
- 3 $(lat, cost, sel) \leftarrow \text{RUN}(p, \mathcal{D}_2)$;
- 4 $acc \leftarrow \text{ACCURACY}(sel, G)$;
- 5 $\mathcal{F} \leftarrow \text{PARETOFRONTIER}(\mathcal{P}, lat, cost)$;
- 6 $p^* \leftarrow \arg \min_{p \in \mathcal{F}} \mathcal{M}[p].lat$;
- 7 **return** p^*

then examines pairwise predicate correlations using a precomputed Matthews Correlation Coefficient (MCC) matrix. It retains only those predicate pairs whose correlation exceeds a threshold μ_{thres} in $\mathcal{F}$ (line 4-7), effectively pruning weakly related combinations. For each retained pair (σ_i, σ_j) , the algorithm applies a fusion function to form a fused predicate $\sigma_i \oplus \sigma_j$ and generates a new candidate path by replacing exactly one occurrence of the original predicates in the sorted sequence with the fused predicate, while preserving the relative order of all remaining predicates (line 8-10). All generated paths are collected into $\mathcal{P}$ and treated as candidate paths for the subsequent path exploration phase.

Algorithm 4 explores the candidate execution paths and selects the best path for the exploitation phase. Given a set of candidate paths $\mathcal{P}$ and a sampled data chunk $\mathcal{D}_2$, the algorithm first executes the reference path p_{ref} to obtain ground-truth results (line 1). Each candidate path is then executed on $\mathcal{D}_2$ to measure its latency, execution cost, and selection output, from which accuracy is computed against the ground truth (lines 2-4). Based on the collected latency and cost metrics, the algorithm constructs a Pareto frontier to retain only non-dominated paths (line 5). Among these Pareto-optimal candidates, the path with the minimum latency or minimum cost is

selected as the final best path p^* and returned for execution (lines 6-7).

C EXPERIMENT RESULTS

D DATASET & BENCHMARK QUERIES

```
SELECT id FROM (
  SELECT TweetID AS id, sentiment, text FROM twitter LIMIT 32768
) WHERE s'The sentiment label rules are:
- sentiment > 0 -> Positive
- sentiment = 0 -> Neutral
- sentiment < 0 -> Negative
Task: Judge whether the {sentiment} label is consistent with the
↪ {text} content.';
```

Figure 14: Q1: Sentiment analysis

```
SELECT s'Classify the given {"Product Name"} into exactly one of
↪ these categories:
[Furniture, Office Supplies, Technology].
Output only the category name, with no explanations, no quotes,
↪ and no extra words.
Example:
Input: Hon Metal Bookcases, Black Output: Furniture
Input: AT&T CL2909 Output: Technology
Input: Belkin F9S820V06 8 Outlet Surge Output: Office Supplies'
↪ AS predicted_category FROM product;
```

Figure 15: Q2: Classify product into one of three categories

Table 8: Query Accuracy for Q1–Q14 (FlockMTL does not support Q1, Q5 and Q20.)

System	Q1				Q2				Q3				Q4			
	Acc.	Pre.	Rec.	F1	Acc.	Pre.	Rec.	F1	Acc.	Pre.	Rec.	F1	Acc.	Pre.	Rec.	F1
Lotus	0.9940	1.0000	0.9940	0.9970	0.7367	0.8148	0.7367	0.7450	0.7157	1.0000	0.7157	0.8343	0.4194	0.8837	0.4194	0.4534
FlockMTL	-	-	-	-	0.7973	0.8221	0.7973	0.7994	0.2294	1.0000	0.2294	0.3732	0.4897	0.7182	0.4897	0.4543
Palimpzest	0.9461	1.0000	0.9461	0.9723	0.3832	0.4471	0.3832	0.4022	0.9426	1.0000	0.9426	0.9705	0.2573	0.4080	0.2573	0.2326
SEMA	0.9934	1.0000	0.9934	0.9967	0.7369	0.8154	0.7369	0.7407	0.7124	1.0000	0.7124	0.8320	0.4428	0.7349	0.4428	0.4878

System	Q5				Q6				Q7				Q8			
	Acc.	Pre.	Rec.	F1	Acc.	Pre.	Rec.	F1	Acc.	Pre.	Rec.	F1	Acc.	Pre.	Rec.	F1
Lotus	0.3040	1.0000	0.3040	0.4662	0.6977	1.0000	0.2561	0.4078	0.8489	0.8722	0.6102	0.7180	0.8542	0.6728	0.8921	0.7670
FlockMTL	-	-	-	-	0.7195	0.9023	0.3475	0.5017	0.7504	0.9558	0.2185	0.3557	0.7445	0.9706	0.0574	0.1083
Palimpzest	0.5799	1.0000	0.5799	0.7341	0.7011	0.9928	0.2664	0.4201	0.8556	0.8316	0.6796	0.7480	0.8644	0.7979	0.6680	0.7272
SEMA	0.1265	1.0000	0.1265	0.2245	0.7030	1.0000	0.2693	0.4243	0.8148	0.7825	0.5715	0.6606	0.8426	0.6958	0.7375	0.7160

System	Q9				Q10				Q11				Q12			
	Acc.	Pre.	Rec.	F1	Acc.	Pre.	Rec.	F1	Acc.	Pre.	Rec.	F1	Acc.	Pre.	Rec.	F1
Lotus	0.9959	0.5000	0.1154	0.1875	0.9456	0.9412	0.3743	0.5356	0.8697	0.9887	0.3954	0.5648	0.7358	0.6534	0.4096	0.5035
FlockMTL	0.9962	1.0000	0.0769	0.1429	0.9251	0.6809	0.1882	0.2949	0.1563	0.0870	0.3103	0.1360	0.6613	0.4722	0.2993	0.3664
Palimpzest	0.9956	0.4167	0.1923	0.2632	0.9407	0.5953	0.9000	0.7166	0.9381	0.8503	0.8621	0.8562	0.6855	0.5168	0.5951	0.5532
SEMA	0.9955	0.3636	0.1538	0.2162	0.9543	0.9364	0.4873	0.6410	0.9651	0.9740	0.8598	0.9133	0.7273	0.6082	0.4683	0.5292

System	Q13				Q14				Q21				Q22			
	Acc.	Pre.	Rec.	F1	Acc.	Pre.	Rec.	F1	Acc.	Pre.	Rec.	F1	Acc.	Pre.	Rec.	F1
Lotus	0.7181	0.8864	0.6754	0.7666	0.7168	0.7767	0.7168	0.7115	0.2683	0.5353	0.3497	0.4230	0.1771	0.1802	0.9107	0.3009
FlockMTL	0.7440	0.8196	0.8035	0.8115	0.7059	0.7684	0.7059	0.7249	0.2743	0.2743	1.0000	0.4305	0.0229	0.0229	1.0000	0.0447
Palimpzest	0.7305	0.8441	0.7445	0.7912	0.6517	0.6763	0.6517	0.6543	0.2635	0.5486	0.3528	0.4312	0.1824	0.2367	0.7014	0.3185
SEMA	0.7315	0.9012	0.6834	0.8883	0.7570	0.7954	0.7570	0.7563	0.2864	0.6477	0.3393	0.4453	0.2017	0.2087	0.8571	0.3357

Table 9: Datasets and Queries Used in the Experiments

Dataset	# Tables	Table (# Rows)	Primary Key	Attributes	Queries
Social_Media [27]	3	twitter (99,901), user (99,260), location (6,211)	twitter.TweetID, user.UserID, location.LocationID	twitter.text	Q1, Q16–Q18
Superstore [27]	1	product (5,298)	product.ProductID	productName, category	Q2
Chicago_Crime [27]	1	IUCR (401)	IUCR.iucr_no	primary_description, secondary_description, index_code	Q3
Food_Inspection [27]	1	violations (36,050)	violations.business_id	description, risk_category	Q4, Q5
CoinMarketCap [7, 11]	2	coins (8,927), historical (4,441,972)	coins.id, historical.coin_id	coins.description	Q6, Q16, Q17
Appstore [23]	2	playstore (10,840), user_reviews (64,295)	playstore.App, user_reviews.App	user_reviews.Translated_Review	Q7–Q9, Q11, Q12
Movies [25, 40, 50]	2	movies (143,258), reviews (1,444,963)	movies.id, reviews.id	reviews.reviewText	Q10, Q13–Q15
Authors [27]	5	Paper (2,254,920), Journal (15,151), Conference (4,545), Author (247,030), PaperAuthor (2,315,574)	Author.Id, Conference.Id, Journal.Id, Paper.Id	Paper.Keyword, Paper.Title	Q19, Q20

```

SELECT iucr_no FROM IUCR
WHERE s'The index_code label rules are:
- index_code = I -> Indexed (severe, such as murder, rape, arson,
  ↳ and robbery)
- index_code = N -> Non-Indexed (less severe, such as vandalism,
  ↳ weapons violations, and peace disturbance)
Task: Determine whether the crime's {primary_description} and
  ↳ {secondary_description} indicates this crime is consisted
  ↳ with {index_code}';

```

Figure 16: Q3: Determine whether the crime’s description is consistent with the index code

```

SELECT s'Task: Classify the given violation {description} into
  ↳ exactly one of these categories:
- Low Risk
- Moderate Risk
- High Risk
Output only the risk category, with no explanations, no quotes,
  ↳ and no extra words.
Example:
Input: Inadequate and inaccessible handwashing facilities.
  ↳ Output: Moderate Risk
Input: Unclean or degraded floors, walls, or ceilings.
Output: Low Risk
Input: Unclean or unsanitary food contact surfaces.
Output: High Risk' AS predicated_category, risk_category AS
  ↳ ground_truth FROM violations;

```

Figure 17: Q4: Determine the risk degree according to the violation description

```

SELECT inspection_id FROM (
  SELECT distinct inspection_id, inspector_comment, fine FROM
  ↪ violation
  WHERE inspector_comment IS NOT NULL
    AND fine IN [100, 250, 500]
  LIMIT 16384
)
WHERE s'The fine for more serious food safety problems will be
↪ higher.
The fine label rules are:
- fine = 100 -> Minor
- fine = 250 -> Serious
- fine = 500 -> Critical
Fine example:
1. {inspector_comment}: REPAIR THE DRAIN STOPPER FOR THE RIGHT
↪ COMPARTMENT OF THE 3 COMPARTMENT SINK SO THAT IT CAN BE USED
↪ TO HOLD WATER. {fine}: 100
2. {inspector_comment}: NO CHEMICAL TEST STRIPS ON PREMISES-MUST
↪ PROVIDE. SERIOUS VIOLATION 7-38-030. {fine}: 250
3. {inspector_comment}: FOUND WALK IN COOLER TEMPERATURE AT 45.3F.
↪ CRITICAL VIOLATION.7-38-005( A) {fine}: 500
Task: Determine whether the {inspector_comment} is consistent
↪ with the {fine}. Output true or false only.';

```

Figure 18: Q5: Determine the risk degree according to the violation description

```

SELECT coins.id AS coin_id FROM coins
WHERE category = 'coin' AND description is NOT NULL
AND s'The {description} shows that this cryptocurrency has a big
↪ supply amount.'
AND s'The {description} indicates that this cryptocurrency is
↪ still in circulation.'
AND s'It can be inferred from the {description} that the price of
↪ this cryptocurrency is rising.'

```

Figure 19: Q6: Find cryptocurrencies that are still in circulation and have a huge supply amount with a rising price

```

SELECT ur.id AS id
FROM user_reviews ur
JOIN playstore p ON ur.App = p.App
WHERE p.Category = 'TOOLS'
AND s'{ur.Translated_Review} is a complete sentence and has
↪ informative content.'
AND s'{ur.Translated_Review} is a positive user review.'
AND s'{ur.Translated_Review} shows that this app is a useful
↪ tool.'

```

Figure 20: Q7: Find positive, complete, and informative user reviews of TOOLS apps that indicate the app is a useful tool

```

SELECT ur.id AS id
FROM user_reviews ur
JOIN playstore p ON ur.App = p.App
WHERE p.Category = 'GAME'
AND s'{Translated_review} is a complete sentence and has
↪ informative content.'
AND s'{Translated_review} is a positive user review.'
AND s'{Translated_Review} shows that users have a good experience
↪ and fun with this app';

```

Figure 21: Q8: Find positive, complete, and informative user reviews of GAME apps that indicate users have a good experience and fun with the app

```

SELECT ur.id AS id
FROM user_reviews ur
JOIN playstore p ON ur.App = p.App
WHERE p.Category = 'FAMILY'
AND s'{Translated_Review} is a complete sentence and has
↪ informative content.'
AND s'{Translated_Review} is a positive user review.'
AND s'{Translated_Review} shows that this app is suitable for
↪ kids.';

```

Figure 22: Q9: Find positive, complete, and informative user reviews of FAMILY apps that indicate the app is suitable for kids

```

SELECT m.movie_id
FROM movie m
WHERE s'The {m.overview} suggests that the movie is an action
↪ movie.'
AND s'The {m.overview} shows that the movie focuses on a central
↪ role.'
AND s'The {m.overview} shows that the movie is based on
↪ historical events';

```

Figure 23: Q10: Find action movies that focus on a central role and are based on historical events

```

SELECT coin_id FROM
(SELECT s'According to {c.description}, introduce the situation
↪ of cryptocurrency circulation' AS circulation, c.id AS
↪ coin_id
FROM coins c JOIN historical h ON h.coin_id = c.id
AND h.date = '2019-01-09'
AND c.status IN ('untracked', 'inactive')
AND c.description IS NOT NULL)
WHERE s'{circulation} indicates cryptocurrency is under
↪ development or in testnet.';

```

Figure 24: Q11: Find cryptocurrencies that were inactive or untracked on Jan 9, 2019, had missing price data, and whose description suggests they were still under development or operating on a testnet

```

SELECT coin_id FROM
(SELECT s'From the description {description}, extract information
↳ about the underlying technology of cryptocurrency, focusing
↳ on whether it mentions a proprietary blockchain, a native
↳ consensus mechanism, or an independent network. Provide a
↳ concise summary of these technological characteristics.' AS
↳ tech_features,
coins.id AS coin_id
FROM coins c JOIN historical h ON h.coin_id = c.id
WHERE c.platform_id IS NOT NULL AND c.description IS NOT NULL
AND h.circulating_supply IS NOT NULL
AND h.total_supply IS NOT NULL
AND h.date = '2019-01-10'
)
WHERE s'Based on the technological characteristics
↳ {tech_features}, determine whether cryptocurrency has its own
↳ independent blockchain or consensus mechanism';

```

Figure 25: Q12: Find coins whose summarized technological features indicate the cryptocurrency has its own blockchain or consensus mechanism

```

SELECT movie_id, s'Classify {plot} into one of the following
↳ categories:
[Sci-Fi, Crime, Thriller, Romance, Drama, Horror, Comedy, War,
↳ Fantasy, Family, Animation, Biography, Other]
Output only the category that best fits the movie, without any
↳ other text or reasoning content.' AS category FROM
(SELECT s'Summarize the main plot of {overview}' AS plot, movie_id
FROM movie m
SEMI JOIN (
SELECT ml.movie_id
FROM movie_languages ml
JOIN language l ON ml.language_id = l.language_id AND
↳ l.language_name = 'English'
) ml ON m.movie_id = ml.movie_id AND movie_status =
↳ 'Released' AND popularity > 10 AND length(overview) < 255
↳ AND length(overview) > 0
);

```

Figure 27: Q14: Summarize the main plot of English-released popular movies and classify plots into genres

```

SELECT movie_id FROM
(SELECT s'Summarize the geographic and temporal background of the
↳ movie according to {overview}.' AS background, m.movie_id AS
↳ movie_id
FROM movie m
SEMI JOIN (
SELECT mg.movie_id
FROM movie_genres mg JOIN genre g ON mg.genre_id =
↳ g.genre_id
WHERE g.genre_name IN ('Thriller', 'Comedy', 'Western')
) mg ON m.movie_id = mg.movie_id
WHERE revenue > budget
AND length(overview) < 255
)
WHERE s'{background} indicates the story is set in modern times
↳ rather than historical periods.';

```

Figure 26: Q13: Find profitable movies whose background indicates the story is set in modern times rather than historical periods with limited genres

```

SELECT movie_id,
s'This is a multi-label classification task. Given the {conflict},
↳ identify what the main character is in conflict with, and
↳ classify it in the format "character vs X", where X is one or
↳ more of: [Character, Self, Society, Nature, Technology,
↳ Machine, Supernatural, Fate, Other].
Output only X, as a comma-separated list (e.g., Self, Society).
Do not include explanations or reasoning.' AS categories
FROM (SELECT movie_id, s'Extract all conflicts from movie
↳ {overview} as one sentence.' AS conflict
FROM movie
SEMI JOIN
(SELECT pc.movie_id
FROM production_country pc JOIN country c
ON pc.country_id = c.country_id AND c.country_iso_code =
↳ 'US')
pc ON movie.movie_id = pc.movie_id AND (movie.release_date
↳ BETWEEN '2000-01-01' AND '2015-12-31') AND
↳ movie.vote_average > 6.0 AND length(overview) < 255;

```

Figure 28: Q15: Extract all conflicts from high-rated movies between 2000 and 2015 and classify conflicts

```

SELECT s'From the job posting tweet {t.text}, extract the
↳ specific job titles and required skills mentioned. Return
↳ them as a comma-separated list (e.g., Software Engineer,
↳ Python, AWS, Machine Learning). Focus on concrete job titles
↳ and technical skills' AS info,t.TweetID AS twitter_id
FROM twitter t JOIN user u
On t.UserID = u.UserID
WHERE user.Gender = 'Female'
AND t.Sentiment = 0.0
AND t.TweetID IS NOT NULL
AND Klout > 50
AND s'Tweets with content as {text} are about job postings,
↳ recruitment announcements, hiring notices, or career
↳ opportunities related to AWS or cloud computing (has contents
↳ like hiring, job opening, career opportunity, we are looking
↳ for, join our team, etc.);

```

Figure 29: Q16: Filter neutral tweets which are posted by females and related to recruitment and extract concrete job titles and technical skills.

```

SELECT s'Based on the tweet content :"{text}", identify the main
↳ type of security or privacy concern being expressed. Classify
↳ text like data breach, unauthorized access, encryption
↳ weakness, compliance violation, identity theft,
↳ service vulnerability, configuration error, monitoring concerns,
↳ or other specific security/privacy concern types.' AS type,
twitter.TweetID AS twitter_id
FROM twitter
WHERE Sentiment < -2.5 AND Klout > 20
AND t.TweetID IS NOT NULL
AND s'Tweet "{text}" discusses security concerns, privacy issues,
↳ data protection, cybersecurity threats, authentication
↳ problems, or confidentiality matters related to AWS
↳ services';

```

Figure 30: Q17:Find negative tweets with high influence that are related to potential risks of AWS services and identify the main type of security or privacy concern

```

SELECT s'From this tweet "{t.text}", extract the main topic or
↳ subject being discussed about programming/development. Return
↳ a concise topic label like performance optimization, learning
↳ resources, job opportunities, technical issues, new features,
↳ etc.'
twitter.TweetID AS twitter_id
FROM twitter t JOIN location l
On t.LocationID = l.LocationID
WHERE l.Country IN ('United States', 'Canada', 'United Kingdom')
AND Sentiment >= 3.0 AND Reach>1000
AND t.TweetID IS NOT NULL
s'Filter tweets "{t.text}" that mention Amazon Web Services or
↳ cloud services, including specific AWS products or general
↳ cloud computing issues';

```

Figure 31: Q18: Filter tweets related to AWS products and extract discussed topics or subjects

```

SELECT sem_agg('Summarize the main research theme reflected by
↳ the following keywords: {p.Keyword}. Respond with a concise,
↳ high-level scientific topic',Keyword) FROM Paper p
JOIN Journal j
ON j.Id= P.JournalId
WHERE p.Year BETWEEN 2000 AND 2011
AND j.ShortName IN ('TODS', 'VLDB', 'TKDE', 'JDM', 'DKE')
AND s'Based on the keywords "{p.Keyword}", determine if this
↳ paper is related to database systems, data management, or
↳ query processing';

```

Figure 32: Q19: Find papers of Database journals and related to database systems, data management, or query processing, and aggregate these papers' keywords to find main research themes

```

SELECT ConferenceId, sem_agg('Identify and summarize the major
↳ interdisciplinary research trends from the following keywords:
↳ {p.Keyword}. Return only one sentence.',Keyword) AS trends
FROM paper p
JOIN Conference c ON p.ConferenceId = c.Id
SEMI JOIN (
SELECT DISTINCT pa.PaperId
FROM PaperAuthor pa
WHERE pa.AuthorId IS NOT NULL
AND (LOWER(COALESCE(pa.Affiliation,'')) LIKE '%institute%'
OR LOWER(pa.Affiliation) LIKE '%laboratory%'
OR LOWER(pa.Affiliation) LIKE '%research center%')
) filtered_papers
ON p.Id = filtered_papers.PaperId
AND p.Keyword IS NOT NULL
AND length(p.Keyword) > 0
WHERE s'Determine whether the paper with keyword as {Keyword} is
↳ related to interdisciplinary methods.'
GROUP BY ConferenceId
HAVING COUNT(*) >= 3 AND trends IS NOT NULL

```

Figure 33: Q20: Find papers whose authors are in institute, laboratory, or research center and identify interdisciplinary research trends

```

SELECT r1.id, r1.reviewId AS review1_id, r2.reviewId AS
↳ review2_id
FROM reviews AS r1
JOIN reviews AS r2
ON r1.id = r2.id
AND r1.reviewId != r2.reviewId
WHERE r1.id = 'action_point' AND r1.scoreSentiment !=
↳ r2.scoreSentiment;

```

Figure 34: Q21:

```

SELECT r1.id, r1.reviewId AS review1_id, r2.reviewId AS
↪ review2_id
FROM reviews AS r1
JOIN reviews AS r2
ON r1.id = r2.id
AND r1.reviewId != r2.reviewId
WHERE r1.id = 'action_point'
AND r1.scoreSentiment = r2.scoreSentiment
AND r1.scoreSentiment = 'POSITIVE';

```

Figure 35: Q22

```

SELECT reviewId AS id
FROM reviews
WHERE id = 'ant_man_and_the_wasp_quantumania'
ORDER BY s'{reviewText} shows the most positive sentiment';

```

Figure 36: Q23

```

SELECT reviewId AS id
FROM reviews
WHERE id = 'ant_man_and_the_wasp_quantumania'
ORDER BY s'{reviewText} shows the most negative sentiment';

```

Figure 37: Q24

```

SELECT reviewId
FROM reviews
WHERE id = 'ant_man_and_the_wasp_quantumania'
ORDER BY s'{reviewText} shows the most positive sentiment' LIMIT
↪ 10;

```

Figure 38: Q25

```

SELECT reviewId
FROM reviews
WHERE id = 'ant_man_and_the_wasp_quantumania'
ORDER BY s'{reviewText} shows the most negative sentiment' LIMIT
↪ 10;

```

Figure 39: Q26

```

SELECT reviewId,
s'Classify the sentiment of {reviewText} into one of [STRONGLY
↪ POSITIVE, POSITIVE, NEUTRAL, NEGATIVE, STRONGLY
↪ NEGATIVE]':::[STRONGLY POSITIVE', 'POSITIVE', 'NEUTRAL',
↪ 'NEGATIVE', 'STRONGLY NEGATIVE'] AS sentiment,
s'Classify the emotion of {reviewText} into one of [EXCITEMENT,
↪ DISAPPOINT, NOSTALGIA, ANGER, CONFUSION,
↪ BOREDOM]':::[EXCITEMENT', 'DISAPPOINT', 'NOSTALGIA', 'ANGER',
↪ 'CONFUSION', 'BOREDOM'] AS emotion,
s'Classify the recommendation intent classification of the
↪ {reviewText} into one of [WOULD RECOMMEND, WOULD NOT
↪ RECOMMEND, RECOMMEND WITH CAVEATS]':::[WOULD RECOMMEND',
↪ 'WOULD NOT RECOMMEND', 'RECOMMEND WITH CAVEATS'] AS
↪ recommendation_intent
FROM reviews
WHERE
id IN ['ant_man_and_the_wasp_quantumania', 'avengers_endgame',
↪ 'captain_marvel', 'avengers_infinity_war'];

```

Figure 40: Q27

```

SELECT reviewId,
s'Classify the sentiment of the reviewer's {reviewText} into one
↪ of [STRONGLY POSITIVE, POSITIVE, NEUTRAL, NEGATIVE, STRONGLY
↪ NEGATIVE]':::[STRONGLY POSITIVE', 'POSITIVE', 'NEUTRAL',
↪ 'NEGATIVE', 'STRONGLY NEGATIVE'] AS sentiment,
s'Classify the recommendation intent classification of the
↪ {reviewText} into one of [WOULD RECOMMEND, WOULD NOT
↪ RECOMMEND, RECOMMEND WITH CAVEATS]':::[WOULD RECOMMEND',
↪ 'WOULD NOT RECOMMEND', 'RECOMMEND WITH CAVEATS'] AS
↪ recommendation_intent,
s'Classify the rewatchability classification of the {reviewText}
↪ into one of [REWATCHABLE, ONE-TIME WATCH, AVOID
↪ ENTIRELY]':::[REWATCHABLE', 'ONE-TIME WATCH', 'AVOID
↪ ENTIRELY'] AS rewatchability
FROM reviews
WHERE
id IN ['ant_man_and_the_wasp_quantumania', 'avengers_endgame',
↪ 'captain_marvel', 'avengers_infinity_war'];

```

Figure 41: Q28

```

SELECT reviewId,
s'Classify the sentiment of the reviewer's {reviewText} into one
↳ of [STRONGLY POSITIVE, POSITIVE, NEUTRAL, NEGATIVE, STRONGLY
↳ NEGATIVE]::[CASE WHEN {reviewText} LIKE '%STRONGLY POSITIVE%' THEN 'STRONGLY POSITIVE'
↳ WHEN {reviewText} LIKE '%POSITIVE%' THEN 'POSITIVE'
↳ WHEN {reviewText} LIKE '%NEUTRAL%' THEN 'NEUTRAL'
↳ WHEN {reviewText} LIKE '%NEGATIVE%' THEN 'NEGATIVE'
↳ WHEN {reviewText} LIKE '%STRONGLY NEGATIVE%' THEN 'STRONGLY NEGATIVE' ELSE NULL END] AS review_quality,
s'Classify the emotion of {reviewText} into one of [EXCITEMENT,
↳ DISAPPOINT, NOSTALGIA, ANGER, CONFUSION,
↳ BOREDOM]::[CASE WHEN {reviewText} LIKE '%EXCITEMENT%' THEN 'EXCITEMENT'
↳ WHEN {reviewText} LIKE '%DISAPPOINT%' THEN 'DISAPPOINT'
↳ WHEN {reviewText} LIKE '%NOSTALGIA%' THEN 'NOSTALGIA'
↳ WHEN {reviewText} LIKE '%ANGER%' THEN 'ANGER'
↳ WHEN {reviewText} LIKE '%CONFUSION%' THEN 'CONFUSION'
↳ WHEN {reviewText} LIKE '%BOREDOM%' THEN 'BOREDOM' ELSE NULL END] AS emotion,
s'Classify the expertise classification of the {reviewText} into
↳ one of [EXPERT, SEMI-EXPERT, CASUAL]::[CASE WHEN {reviewText} LIKE '%EXPERT%' THEN 'EXPERT'
↳ WHEN {reviewText} LIKE '%SEMI-EXPERT%' THEN 'SEMI-EXPERT'
↳ WHEN {reviewText} LIKE '%CASUAL%' THEN 'CASUAL' ELSE NULL END] AS writing_style
FROM reviews
WHERE
id IN ['ant_man_and_the_wasp_quantumania', 'avengers_endgame',
↳ 'captain_marvel', 'avengers_infinity_war'];

```

Figure 42: Q29

```

SELECT reviewId,
s'Classify the rewatchability classification of the {reviewText}
↳ into one of [REWATCHABLE, ONE-TIME WATCH, AVOID
↳ ENTIRELY]::[CASE WHEN {reviewText} LIKE '%REWATCHABLE%' THEN 'REWATCHABLE'
↳ WHEN {reviewText} LIKE '%ONE-TIME WATCH%' THEN 'ONE-TIME WATCH'
↳ WHEN {reviewText} LIKE '%AVOID ENTIRELY%' THEN 'AVOID ENTIRELY' ELSE NULL END] AS rewatchability,
s'Classify the emotion of {reviewText} into one of [EXCITEMENT,
↳ DISAPPOINT, NOSTALGIA, ANGER, CONFUSION,
↳ BOREDOM]::[CASE WHEN {reviewText} LIKE '%EXCITEMENT%' THEN 'EXCITEMENT'
↳ WHEN {reviewText} LIKE '%DISAPPOINT%' THEN 'DISAPPOINT'
↳ WHEN {reviewText} LIKE '%NOSTALGIA%' THEN 'NOSTALGIA'
↳ WHEN {reviewText} LIKE '%ANGER%' THEN 'ANGER'
↳ WHEN {reviewText} LIKE '%CONFUSION%' THEN 'CONFUSION'
↳ WHEN {reviewText} LIKE '%BOREDOM%' THEN 'BOREDOM' ELSE NULL END] AS emotion,
s'Classify the recommendation intent classification of the
↳ {reviewText} into one of [WOULD RECOMMEND, WOULD NOT
↳ RECOMMEND, RECOMMEND WITH CAVEATS]::[CASE WHEN {reviewText} LIKE '%WOULD RECOMMEND%' THEN 'WOULD RECOMMEND'
↳ WHEN {reviewText} LIKE '%WOULD NOT RECOMMEND%' THEN 'WOULD NOT RECOMMEND'
↳ WHEN {reviewText} LIKE '%RECOMMEND WITH CAVEATS%' THEN 'RECOMMEND WITH CAVEATS' ELSE NULL END] AS
↳ recommendation_intent,
FROM reviews
WHERE
id IN ['ant_man_and_the_wasp_quantumania', 'avengers_endgame',
↳ 'captain_marvel', 'avengers_infinity_war'];

```

Figure 44: Q31

```

SELECT id,
s'According to {description}, classify the investment value of
↳ the cryptocurrency into one of the following categories
↳ [BULLISH, BEARISH, NEUTRAL]::[CASE WHEN {description} LIKE '%BULLISH%' THEN 'BULLISH'
↳ WHEN {description} LIKE '%BEARISH%' THEN 'BEARISH'
↳ WHEN {description} LIKE '%NEUTRAL%' THEN 'NEUTRAL' ELSE NULL END] AS investment_value
FROM coins
WHERE category='coin'
AND s'The {description} indicates this cryptocurrency is in
↳ circulation, has active trading, and is tracked by market
↳ data platforms'
AND s'The {description} shows that the price of cryptocurrency is
↳ not high.';

```

Figure 45: Q32

```

SELECT reviewId,
s'Classify the sentiment of the reviewer's {reviewText} into one
↳ of [STRONGLY POSITIVE, POSITIVE, NEUTRAL, NEGATIVE, STRONGLY
↳ NEGATIVE]::[CASE WHEN {reviewText} LIKE '%STRONGLY POSITIVE%' THEN 'STRONGLY POSITIVE'
↳ WHEN {reviewText} LIKE '%POSITIVE%' THEN 'POSITIVE'
↳ WHEN {reviewText} LIKE '%NEUTRAL%' THEN 'NEUTRAL'
↳ WHEN {reviewText} LIKE '%NEGATIVE%' THEN 'NEGATIVE'
↳ WHEN {reviewText} LIKE '%STRONGLY NEGATIVE%' THEN 'STRONGLY NEGATIVE' ELSE NULL END] AS review_quality,
s'Classify the emotion of {reviewText} into one of [EXCITEMENT,
↳ DISAPPOINT, NOSTALGIA, ANGER, CONFUSION,
↳ BOREDOM]::[CASE WHEN {reviewText} LIKE '%EXCITEMENT%' THEN 'EXCITEMENT'
↳ WHEN {reviewText} LIKE '%DISAPPOINT%' THEN 'DISAPPOINT'
↳ WHEN {reviewText} LIKE '%NOSTALGIA%' THEN 'NOSTALGIA'
↳ WHEN {reviewText} LIKE '%ANGER%' THEN 'ANGER'
↳ WHEN {reviewText} LIKE '%CONFUSION%' THEN 'CONFUSION'
↳ WHEN {reviewText} LIKE '%BOREDOM%' THEN 'BOREDOM' ELSE NULL END] AS emotion,
s'Classify the recommendation intent classification of the
↳ {reviewText} into one of [WOULD RECOMMEND, WOULD NOT
↳ RECOMMEND, RECOMMEND WITH CAVEATS]::[CASE WHEN {reviewText} LIKE '%WOULD RECOMMEND%' THEN 'WOULD RECOMMEND'
↳ WHEN {reviewText} LIKE '%WOULD NOT RECOMMEND%' THEN 'WOULD NOT RECOMMEND'
↳ WHEN {reviewText} LIKE '%RECOMMEND WITH CAVEATS%' THEN 'RECOMMEND WITH CAVEATS' ELSE NULL END] AS
↳ recommendation_intent,
FROM reviews
WHERE
id IN ['ant_man_and_the_wasp_quantumania', 'avengers_endgame',
↳ 'captain_marvel', 'avengers_infinity_war'];

```

Figure 43: Q30

```

SELECT id,
s'According to {description}, classify the investment value of
↳ the cryptocurrency into one of the following categories
↳ [BULLISH, BEARISH, NEUTRAL]::[CASE WHEN {description} LIKE '%BULLISH%' THEN 'BULLISH'
↳ WHEN {description} LIKE '%BEARISH%' THEN 'BEARISH'
↳ WHEN {description} LIKE '%NEUTRAL%' THEN 'NEUTRAL' ELSE NULL END] AS investment_value
FROM coins
WHERE category='coin'
AND s'The {description} indicates this cryptocurrency is in
↳ circulation, has active trading, and is tracked by market
↳ data platforms'
AND s'The {description} mentions the last known price of
↳ cryptocurrency.';

```

Figure 46: Q33

```

SELECT reviewId,
s'Classify the emotion of {reviewText} into one of [EXCITEMENT,
↳ DISAPPOINT, NOSTALGIA, ANGER, CONFUSION,
↳ BOREDOM]::[EXCITEMENT', 'DISAPPOINT', 'NOSTALGIA', 'ANGER',
↳ 'CONFUSION', 'BOREDOM'] AS emotion,
s'Classify the recommendation intent classification of the
↳ {reviewText} into one of [WOULD RECOMMEND, WOULD NOT
↳ RECOMMEND, RECOMMEND WITH CAVEATS]::[WOULD RECOMMEND',
↳ 'WOULD NOT RECOMMEND', 'RECOMMEND WITH CAVEATS'] AS
↳ recommendation_intent
FROM reviews
WHERE id IN ['ant_man_and_the_wasp_quantumania',
↳ 'avengers_endgame', 'captain_marvel',
↳ 'avengers_infinity_war']
AND s'The sentiment of the reviewer's {reviewText} is positive');

```

Figure 47: Q34

```

SELECT reviewId,
s'Classify the recommendation intent classification of the
↳ {reviewText} into one of [WOULD RECOMMEND, WOULD NOT
↳ RECOMMEND, RECOMMEND WITH CAVEATS]::[WOULD RECOMMEND',
↳ 'WOULD NOT RECOMMEND', 'RECOMMEND WITH CAVEATS'] AS
↳ recommendation_intent,
FROM reviews
WHERE
id IN ['ant_man_and_the_wasp_quantumania', 'avengers_endgame',
↳ 'captain_marvel', 'avengers_infinity_war']
AND s'The sentiment of the reviewer's {reviewText} is positive'
AND s'The emotion of {reviewText} is excitement or nostalgia';

```

Figure 48: Q35

```

COPY (SELECT reviewId,
s'Classify the sentiment of {reviewText} into one of [STRONGLY
↳ POSITIVE, POSITIVE, NEUTRAL, NEGATIVE, STRONGLY
↳ NEGATIVE]::[STRONGLY POSITIVE', 'POSITIVE', 'NEUTRAL',
↳ 'NEGATIVE', 'STRONGLY NEGATIVE'] AS sentiment,
s'Classify the recommendation intent classification of the
↳ {reviewText} into one of [WOULD RECOMMEND, WOULD NOT
↳ RECOMMEND, RECOMMEND WITH CAVEATS]::[WOULD RECOMMEND',
↳ 'WOULD NOT RECOMMEND', 'RECOMMEND WITH CAVEATS'] AS
↳ recommendation_intent,
FROM reviews
WHERE
id IN ['ant_man_and_the_wasp_quantumania', 'avengers_endgame',
↳ 'captain_marvel', 'avengers_infinity_war']
AND s'The emotion of {reviewText} is excitement or nostalgia') TO
↳ 'q20.csv' WITH (FORMAT CSV, HEADER);

```

Figure 49: Q36

```

-- Note: Introduces redundant expressions - repeated field
↳ references ({primary_description}, {secondary_description},
↳ {index_code} each appear twice), redundant modifiers ("after
↳ reviewing...again for consistency"), and complex sentence
↳ structures.
SELECT COUNT(*) FROM chicago_crime.IUCR
WHERE s'The index_code label rules are:\n\
- index_code = I -> Indexed (severe, such as murder, rape, arson,
↳ and robbery)\n\
- index_code = N -> Non-Indexed (less severe, such as vandalism,
↳ weapons violations, and peace disturbance)\n\
Based on these rules, the crime with primary description
↳ {primary_description} and secondary description
↳ {secondary_description} should be checked against index code
↳ {index_code}, and after reviewing the primary description
↳ {primary_description} again for consistency, it should be
↳ determined whether this crime is consistent with the assigned
↳ index code {index_code} according to the established
↳ classification criteria.';

```

Figure 50: Q3 before Expression Reduction

```

SELECT iucr_no FROM IUCR
WHERE s'The index_code label rules are:
- index_code = I -> Indexed (severe, such as murder, rape, arson,
↳ and robbery)
- index_code = N -> Non-Indexed (less severe, such as vandalism,
↳ weapons violations, and peace disturbance)
Task: Determine whether the crime's {primary_description} and
↳ {secondary_description} indicates this crime is consisted
↳ with {index_code}';

```

Figure 51: Q3 before Partial Deduction

```

-- Note: Introduces repeated field references ({description}
↳ appears three times) and redundant sentence structures
↳ (repetitive "and the {description} indicates that..."
↳ patterns).
SELECT coins.id AS coin_id FROM coins
WHERE category = 'coin' AND status = 'active' AND description is
↳ NOT NULL
AND s'The {description} shows that this cryptocurrency has a big
↳ supply amount, and the {description} indicates that this
↳ cryptocurrency is still in circulation, and the {description}
↳ can be used to infer that the price of this cryptocurrency is
↳ rising.';

```

Figure 52: Q6 before Expression Reduction

```

SELECT coins.id AS coin_id FROM coins
WHERE category = 'coin' AND description IS NOT NULL
AND s'The {description} shows that this cryptocurrency has a big
↪ supply amount.'
AND s'The {description} indicates that this cryptocurrency is
↪ still in circulation.'
AND s'It can be inferred from the {description} that the price of
↪ this cryptocurrency is rising.'

```

Figure 53: Q6 before Partial Deduction

```

-- Note: Adds redundant modifiers ("which describes the film
↪ narrative"), repeated field references ({overview} appears
↪ twice), and complex expressions ("after reviewing...for
↪ temporal clues").
SELECT movie_id FROM
  (SELECT s'Based on the movie overview {overview} provided,
    ↪ which describes the film narrative, summarize the
    ↪ geographic and temporal background of the movie,
    ↪ including specific locations and time periods mentioned
    ↪ in this overview {overview}.' AS background, movie_id
    FROM movie
    SEMI JOIN (
      SELECT mg.movie_id
      FROM movie_genres mg JOIN genre g ON mg.genre_id =
        ↪ g.genre_id
      WHERE g.genre_name IN ('Thriller', 'Comedy', 'Western')
    ) mg ON m.movie_id = mg.movie_id
    WHERE revenue > budget
    AND length(overview) < 255
  )
WHERE s'The background summary {background} indicates the
↪ story is set in modern times rather than historical
↪ periods, and after reviewing the background {background}
↪ for temporal clues, this setting classification is
↪ confirmed.';

```

Figure 54: Q13 before Expression Reduction

```

-- Note: Introduces lengthy classification descriptions and
↪ repeated field references ({plot} appears three times), adds
↪ unnecessary explanatory text.
SELECT plot,s'For genre classification purposes, the plot summary
↪ {plot} should be examined, and following a review of this plot
↪ {plot}, assignment to one of the predefined categories is
↪ required, including: Sci-Fi, Crime/Thriller, Romance/Drama,
↪ Horror, Comedy, War, Fantasy, Family/Animation, Biography, or
↪ if none match, classification as Other.' AS category
FROM
  (SELECT s'After reading the movie overview {overview}, which
    ↪ provides a description of the film, summarization of the main
    ↪ plot points and key narrative elements from this overview
    ↪ {overview} should be performed in a concise manner.' AS plot
    FROM movie m
    SEMI JOIN (
      SELECT ml.movie_id
      FROM movie_languages ml
      JOIN "language" l ON ml.language_id = l.language_id AND
        ↪ l.language_name = 'English'
    ) ml ON m.movie_id = ml.movie_id AND movie_status =
    ↪ 'Released' AND popularity > 10 AND length(overview) < 255
    AND length(overview) > 0
  );

```

Figure 56: Q14 before Expression Reduction

```

-- Note: Replaces originally SQL-expressible numerical comparison
↪ predicate revenue > budget with natural language description
↪ 'the {revenue} is greater than {budget}'.
SELECT movie_id FROM
  (SELECT s'Summarize the geographic and temporal background of the
    ↪ movie according to {overview}.' AS background, movie_id
    FROM movie
    SEMI JOIN (
      SELECT mg.movie_id
      FROM movie_genres mg JOIN genre g ON mg.genre_id =
        ↪ g.genre_id
      WHERE g.genre_name IN ('Thriller', 'Comedy', 'Western')
    ) mg ON m.movie_id = mg.movie_id
    WHERE s'the {revenue} is greater than {budget}'
    AND length(overview) < 255
  )
WHERE s'{background} indicates the story is set in modern times
↪ rather than historical periods.';

```

Figure 55: Q13 before Predicate Deduction

```

-- Note: Replaces SQL-expressible movie_status = 'Released' with
↪ originally natural language description '{movie_status} is
↪ released'.
SELECT plot,s'Classify {plot} into one of: Sci-Fi, Crime,
↪ Thriller, Romance, Drama, Horror, Comedy, War, Fantasy,
↪ Family, Animation, Biography, Other' AS category
FROM
  (SELECT s'Summarize the main plot of {overview}' AS plot
    FROM movie m
    SEMI JOIN (
      SELECT ml.movie_id
      FROM movie_languages ml
      JOIN "language" l ON ml.language_id = l.language_id AND
        ↪ l.language_name = 'English'
    ) ml ON m.movie_id = ml.movie_id AND s'{movie_status} is
    ↪ released' AND popularity > 10 AND length(overview) < 255
    AND length(overview) > 0
  );

```

Figure 57: Q14 before Predicate Deduction

E SYSTEM PROMPT DETAILS

Prompt used for partial deduction

You are a database optimizer assistant. Your task is to deduce the necessary SQL preconditions (pushdown predicates) for natural language expressions (NLEs), based on the provided column statistics.

An NLE is evaluated by an LLM to determine if a row satisfies a certain condition. Because LLM evaluation is expensive, we want to filter out rows that are guaranteed to be false by executing simple SQL predicates on the CPU first.

IMPORTANT: Your goal is NOT to convert the NLE into an equivalent SQL statement. Instead, you must find the necessary conditions of the NLE predicate given the column statistics.

The provided statistics for each column include:

- whether it is nullable
- the number of distinct values
- the top-5 most frequent values (sampled from a subset of rows)

Steps:

1. Understand the semantic meaning of the NLE.
2. Examine the column statistics, especially the top frequent values.
3. Identify values that definitively violate the NLE predicate.

For instance, nan, N/A, empty strings, or NULL often represent missing data.

If the NLE predicate requires a valid or meaningful value, these missing values can be filtered out.

4. Formulate DuckDB SQL predicates that filter out these violating values.

If you can deduce necessary conditions, output them as a valid JSON array of strings.

Each string must be a valid DuckDB SQL predicate (e.g., "col != 'nan'").

The final filter will be the logical AND of all strings in the array.

If no meaningful necessary condition can be deduced, simply output an empty array.

Example:

NLE Predicate:

Translated_review is a positive user review

Column stats:

Translated_Review: nullable=true, distinct=10537,
top5=["nan":711, "Good":13, "Negative":10,
"Great":6, "Really good":5]

Output:

```
["Translated\_Review != 'nan'", "Translated\_Review  
!= 'Negative'"]
```

Note:

The output MUST be a valid JSON array of strings (or an empty array).

Prompt used for partial deduction verification

You are a database optimizer assistant. Your task is to verify whether each candidate SQL predicate is a NECESSARY CONDITION for a set of Natural Language Expressions (NLEs).

Background:

A semantic filter evaluates NLEs (natural language conditions) on each row. Because LLM evaluation is expensive, we pre-filter rows using cheap SQL predicates. A SQL predicate P is a valid pushdown filter only if it is a NECESSARY CONDITION: whenever P is false, the overall NLE filter is guaranteed to be false.

KEY CONCEPT - Necessary vs Sufficient:

For each column, some values make NLE FALSE (set A), some make it TRUE (set B).

- **Necessary (valid)**: `"column != value"` for values in A. Filtering out violating values is correct.
- **Sufficient (invalid)**: `"column = value"` or `"column IN (...)"` for values in B. Whitelisting known-good values is WRONG: it would exclude valid rows whose values are not in the statistics. Output false for such predicates.

Definition of necessary condition:

SQL predicate P is a necessary condition for NLE set S iff:

for every possible row, $\text{NOT } P \Rightarrow$ (at least one NLE in S is false)

Equivalently: if a row passes all NLEs, it must also satisfy P.

Given the original NL Expressions and the candidate SQL Predicates, output a JSON array of booleans (same length as the SQL Predicates list):

- true: the SQL predicate IS a valid necessary condition (typically `"column != value"` filtering out violating values)
- false: the SQL predicate is NOT valid (too strict, unrelated to NLEs, or a sufficient condition like `"column = value"` / `"column IN (...)"`)

Example:\n"

NL Expressions: `["{Review} is a positive and meaningful user review"]`

SQL Predicates: `["Review != 'nan'", "Review = 'Good'", "Rating > 10"]`

Output: `[true, false, false]`

(Review != 'nan' is necessary: 'nan' violates the NLE. Review = 'Good' is NOT necessary: it is a sufficient condition that would wrongly exclude other positive reviews. Rating > 10 is unrelated to the NLE.)

Note: Output MUST be a valid JSON array of booleans only.;